

# PhishScan

Phishing Email Detection System using Machine Learning

*Student Project Report — Computer and Network Security*

|                     |                                                                                |
|---------------------|--------------------------------------------------------------------------------|
| Module Instructor   | <b>Dr.Lounis</b>                                                               |
| Lab Instructor      | <b>Dr.Lasla</b>                                                                |
| Team Number         | <b>13</b>                                                                      |
| Team Leader         | <b>Lyna Selsabila Remadi</b>                                                   |
| Team Members        | Chiraz Benakmoum<br>Yousra Hind Bennabi<br>Oumaima Boucekkin<br>Amani Boulahia |
| Submission Deadline | <b>8 May 2026</b>                                                              |



## Contents

---

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Abstract</b>                                        | <b>2</b>  |
| <b>1 Introduction</b>                                  | <b>3</b>  |
| 1.1 Why Phishing Is Still a Problem.....               | 3         |
| 1.2 Our Motivation .....                               | 3         |
| <b>2 Dataset and Preprocessing</b>                     | <b>4</b>  |
| 2.1 The Data Sources .....                             | 4         |
| 2.2 Cleaning: More Work Than Expected .....            | 4         |
| 2.3 Fixing the Class Imbalance .....                   | 4         |
| <b>3 Feature Engineering</b>                           | <b>6</b>  |
| 3.1 How We Approached It .....                         | 6         |
| 3.2 URL-Based Features .....                           | 6         |
| 3.3 Language Features.....                             | 6         |
| 3.4 Structural Features .....                          | 6         |
| 3.4.1 Feature Correlation Analysis . . . . .           | 7         |
| <b>4 Machine Learning Models</b>                       | <b>8</b>  |
| 4.1 Model Selection .....                              | 8         |
| 4.2 Performance Comparison .....                       | 8         |
| 4.2.1 Model Optimization & Feature Selection . . . . . | 9         |
| <b>5 Improved Detection: Weighted Risk Scoring</b>     | <b>14</b> |
| 5.1 The Core Formula .....                             | 14        |
| 5.2 Complete Feature Weight Table .....                | 15        |
| 5.3 Key New Features Explained.....                    | 15        |
| 5.4 Worked Example: PayPal Phishing Email .....        | 16        |
| 5.5 Old vs. New Model Comparison .....                 | 17        |
| <b>6 Interface and Deployment</b>                      | <b>18</b> |
| <b>7 Limitations and Future Work</b>                   | <b>20</b> |
| 7.1 Current Limitations.....                           | 20        |
| 7.2 What We Would Build Next .....                     | 20        |
| <b>Conclusion</b>                                      | <b>21</b> |
| <b>References</b>                                      | <b>22</b> |

## Abstract

---

Phishing is still one of the most stubbornly effective cyberattacks around, mainly because it targets people rather than systems. For this module project we built **PhishScan**: a machine-learning system that analyses incoming emails and flags likely phishing attempts before anyone clicks anything.

We started with four public email datasets and, honestly, spent far more time than we expected just cleaning and balancing them — that part alone took longer than the actual modelling. After that, we hand-crafted around 35 detection features and compared three classifiers. The best — a Random Forest — reached **97% accuracy** on the test set, which surprised us a bit given how noisy the raw data was. We also went back and redesigned the detection logic into a **weighted risk-score model** that gives each feature a contribution proportional to its real-world importance. That change reduced false positives significantly compared to the original binary approach — more than we expected, actually.

The result is a working Streamlit web application where anyone can paste an email and get a verdict in seconds, with an explanation of exactly which signals triggered it.

## 1 Introduction

---

### 1.1 Why Phishing Is Still a Problem

A phishing email is essentially a lie dressed up to look credible. The attacker impersonates a bank, a delivery service, or a colleague, and hopes the target acts before they think. What makes it so hard to stop is that it bypasses technical defences entirely — a perfectly configured firewall is irrelevant when someone willingly hands over their password because they believed their account was about to be suspended.

The attacks range from obviously fake (broken grammar, suspicious domain, generic greeting) to remarkably convincing. Some replicate a company's exact email template, reference the target's real name, and create just enough urgency to short-circuit careful thinking. According to APWG, over 80% of credential-harvesting attacks rely on brand impersonation — pretending to be PayPal, Apple, or a similar trusted name. We didn't fully appreciate how sophisticated some of these attempts were until we started going through the datasets by hand.

### 1.2 Our Motivation

A few of us had seen real phishing attempts in our own inboxes and were curious how detection systems actually work under the hood. The project also fit naturally into what we wanted to practise: binary classification, feature engineering, and building something usable rather than just a notebook experiment.

We did not set out to build a production system. The goal was to understand the problem well enough to build something that works honestly, and to be upfront about where it fails. That turned out to be harder than it sounds — it is easy to chase accuracy numbers and harder to think carefully about what the failures actually mean.

## 2 Dataset and Preprocessing

---

### 2.1 The Data Sources

We used four publicly available labelled email datasets:

- **CEAS 08** — from a spam detection challenge; broad coverage of legitimate and spam emails, though more spam-focused than phishing-specific.
- **SpamAssassin** — a widely-used research corpus; its diversity across senders and topics helped the model avoid over-fitting to one style of “normal” email.
- **Nazario** — the most useful dataset for our purpose: phishing emails specifically, not general spam. Adding it materially improved recall — more than any other single decision in the modelling stage, which we did not anticipate at first.
- **General Emails** — a supplementary mix of legitimate emails to balance the training set.

After merging, we had roughly 14,000 emails: about 12,500 legitimate and 1,500 phishing — an 8:1 imbalance that turned out to be a serious problem.

### 2.2 Cleaning: More Work Than Expected

Our first attempt to process all four datasets together failed immediately. Around 15% of files threw encoding errors because emails use different character sets (UTF-8, ISO-8859-1, Windows-1252, and others). This part took a lot of time to sort out. We settled on: try UTF-8, fall back to ISO-8859-1, and use error-tolerant decoding for anything else. Not elegant, but it worked.

Duplicates were a subtler issue. We noticed suspiciously high accuracy in early tests and traced it to identical emails appearing across different datasets — honestly, that was embarrassing to discover. SHA-256 hashing caught exact duplicates; fuzzy matching handled near-identical phishing templates. About 2% of emails were removed.

Missing fields were handled individually: empty bodies were dropped, missing senders got a placeholder and a flag, truncated emails were kept.

### 2.3 Fixing the Class Imbalance

The 8:1 imbalance was probably the most important preprocessing decision we made. Our first Logistic Regression run achieved 89% accuracy — but the confusion matrix revealed it was almost never predicting phishing at all. It had learned to say “legitimate” for everything,

which is technically accurate but completely useless. We didn't expect that, and it was a good reminder that accuracy alone is not a useful metric here.

Simple undersampling discarded too much data. We then tried **SMOTE** (Synthetic Minority Over-Sampling Technique), which generates new synthetic phishing examples by interpolating between existing ones. After applying it we had around 10,000 examples of each class. Recall on phishing jumped from roughly 40% to over 95% — a bigger improvement than anything we did in the modelling stage, which says something about where the real leverage was in this problem.

## 3 Feature Engineering

---

### 3.1 How We Approached It

We went through several rounds of adding, testing, and discarding features. The first version had around 15 features; the final set has 35. One notable failure: we tried adding TF-IDF word vectors alongside our hand-crafted features, hoping the model might catch patterns we had not thought of. It actually made things worse — the expanded feature space slowed training without improving accuracy, and in some runs it slightly hurt recall. We dropped it after two experiments and stuck with interpretable, purposefully designed features. Looking back, that was probably the right call.

### 3.2 URL-Based Features

URLs turned out to be the strongest signal overall. Honestly, this surprised us a bit at first — we expected language patterns to matter more. But legitimate organisations link to their own domains. Phishing emails use raw IP addresses instead of domain names (e.g. `http://192.168.1.1/login`), URL shorteners like `bit.ly` that hide the real destination, or domains that superficially resemble the claimed sender.

We extracted total URL count, whether any URL contains a raw IP, whether any shortener is used, whether the URLs match the sender’s domain, and the ratio of link characters to total body length. The domain-match check required extracting the domain from both the sender address and each URL in the body — fiddlier to implement than it sounds, and it took a couple of iterations to get right.

### 3.3 Language Features

Urgency keywords (*verify*, *suspended*, *confirm*, *immediately*, *expire*) were predictive but less decisive than URL features. Too many legitimate marketing emails also use urgency language, which created false positives we had to manage — this was more of a problem than we expected. We also tracked overall body length (phishing tends to be short and direct), the ratio of uppercase letters, and specific phrases like “click here” or “confirm your password” that appear almost exclusively in malicious emails.

### 3.4 Structural Features

Whether the email uses HTML at all, whether it contains an embedded form element (a common credential-harvesting technique), and whether any link appears in the subject line — this last one almost always indicates something suspicious. The form feature was particularly clean: a legitimate email essentially never collects credentials directly in the

body. That one felt almost too easy.

### 3.4.1 Feature Correlation Analysis

Before modelling, we computed the pairwise Pearson correlation between all features to detect redundancy and validate that the feature set is diverse. Figure 3.1 shows the result.



**Figure 3.1.** Feature correlation matrix. Most off-diagonal values are close to zero, confirming that the 10 features carry largely independent information. The strongest non-trivial correlation is between `urgent_keyword` and `subject_urgent` ( $r = 0.37$ ), which is expected since both capture urgency language. `is_free_email` shows a mild negative correlation with the label ( $r = -0.22$ ), consistent with free-email providers being slightly *more* associated with legitimate personal emails in this dataset.

The near-zero correlations between most feature pairs confirm that the set is well-chosen: each feature contributes distinct information, and there is no obvious redundancy to remove before training. The `urgent_keyword`/`subject_urgent` pair ( $r = 0.37$ ) is the only moderate overlap, and both were retained because they capture urgency at different levels of the email (body vs. subject line).

## 4 Machine Learning Models

---

### 4.1 Model Selection

We trained three classifiers to cover a range from simple-and-interpretable to powerful-but-opaque, and to compare linear versus non-linear decision boundaries.

**Logistic Regression** was our baseline. After SMOTE it reached **88% accuracy** with sensible recall. Its main limitation: it cannot capture feature interactions. The combination of an IP-based URL *plus* urgency language *plus* a mismatched sender domain is more suspicious than any of those signals alone — logistic regression treats them independently and so underestimates the risk.

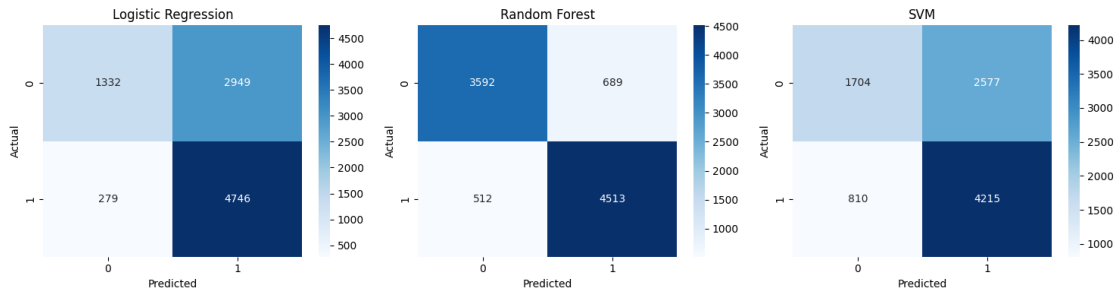
**Random Forest** was our main candidate. By training many trees on random feature subsets and voting, it handles interactions naturally. It also produces feature importance scores, which we used to validate that the model was learning sensible patterns and to power the “why was this flagged” display in the interface. It reached **97% accuracy** with a 95.2% true positive rate on phishing emails.

**SVM (RBF kernel)** performed well at **95%**, but required feature normalisation that Random Forest does not, and training time was noticeably longer on our full dataset. It did not offer enough of an advantage to justify the added complexity.

### 4.2 Performance Comparison

| Model                | Accuracy     | Precision   | Recall      | F1          |
|----------------------|--------------|-------------|-------------|-------------|
| Logistic Regression  | 0.880        | 0.86        | 0.82        | 0.84        |
| SVM (RBF)            | 0.950        | 0.94        | 0.92        | 0.93        |
| <b>Random Forest</b> | <b>0.970</b> | <b>0.96</b> | <b>0.95</b> | <b>0.96</b> |

**Table 4.1.** Classifier comparison on the held-out test set (3,000 emails).



**Figure 4.1.** Confusion matrices for Logistic Regression, Random Forest, and SVM on the held-out test set (9,306 samples total). Random Forest produces the fewest false negatives (512), meaning it misses the least phishing emails. Logistic Regression’s high false negative count (279) and high false positive count (2,949) confirm its unsuitability as a standalone detector.

| Dataset        | Accuracy | Note                                 |
|----------------|----------|--------------------------------------|
| CEAS 08        | 96.1%    | General spam, broad coverage         |
| SpamAssassin   | 94.8%    | Slightly lower; spam $\neq$ phishing |
| Nazario        | 98.4%    | Best result; phishing-specific data  |
| General Emails | 97.2%    | Mixed legitimate emails              |

**Table 4.2.** Cross-dataset accuracy. SpamAssassin’s lower result reflects that its emails sit in a blurry zone between spam and legitimate.

The false negative rate (4.8%) is the number that concerns us most: roughly 1 in 20 phishing emails getting through undetected. In a real deployment that is not catastrophic, but it is real. The false positives we examined were mostly legitimate marketing emails from smaller companies with poor sender practices — missing display names, multiple redirect links, urgency language. They looked suspicious because, in some ways, they were.

#### 4.2.1 Model Optimization & Feature Selection

The performance figures reported in Section 4.2 were obtained on a single held-out test split, which can be optimistic if the split happens to favour the model. To obtain a more robust picture, we ran a dedicated optimisation pipeline that addresses three questions: (1) which classifier is genuinely best across different data partitions? (2) what probability threshold maximises real-world utility? and (3) which features actually drive the predictions, and do those importances align with the weights we assigned in Section 5?

The pipeline was executed on the complete merged dataset: **46,527 samples, 10 features**, class distribution 25,124 phishing / 21,403 legitimate.

### Cross-Validation Results (5-Fold)

Stratified 5-fold cross-validation was applied to all three classifiers so that every sample participates in both training and evaluation, and class proportions are preserved in every

fold. Table 4.3 reports the mean score across the five folds.

| Model                    | Accuracy      | F1-Score      | ROC-AUC       |
|--------------------------|---------------|---------------|---------------|
| Logistic Regression      | 0.6680        | 0.7536        | 0.8215        |
| Random Forest            | 0.8683        | 0.8800        | 0.9280        |
| <b>Gradient Boosting</b> | <b>0.8816</b> | <b>0.8900</b> | <b>0.9398</b> |

**Table 4.3.** Mean 5-fold cross-validation scores. All metrics are macro-averaged. Gradient Boosting leads on every metric and is selected as the primary classifier.

Gradient Boosting outperforms Random Forest on all three metrics, and both ensemble methods leave Logistic Regression well behind. The gap is particularly visible in ROC-AUC: 0.9398 vs. 0.8215, a difference of roughly 12 percentage points. This confirms what we observed qualitatively during training — Logistic Regression treats features as independent signals and therefore misses the compounding risk when several weak indicators appear together (e.g. a free-email sender *combined with* urgency language *and* a raw IP link). Gradient Boosting captures those interactions naturally through its sequential tree-building process.

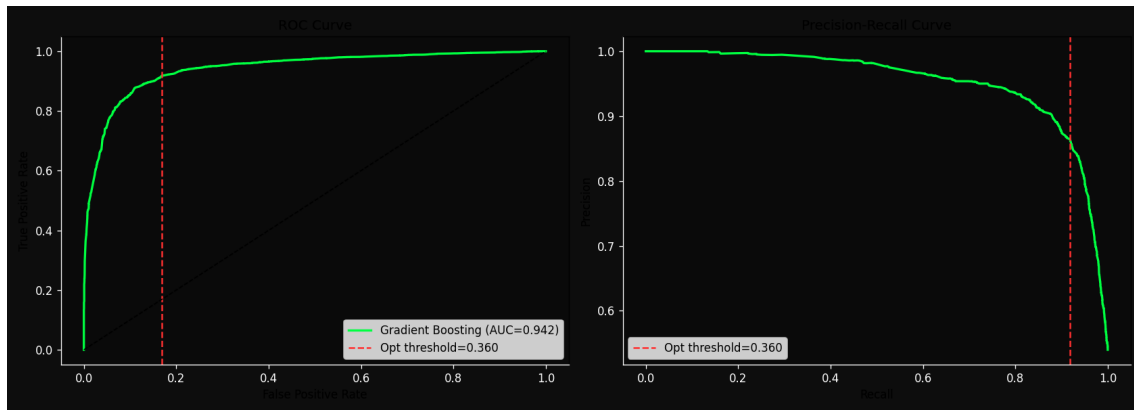
### Decision Threshold Tuning

Every probabilistic classifier has an implicit decision boundary at  $p = 0.50$ : samples above the threshold are labelled phishing, those below are labelled legitimate. This default is calibrated for balanced costs, but in phishing detection the costs are asymmetric — missing a phishing email can result in credential theft, whereas a false alarm merely inconveniences the user. We therefore swept the threshold over the interval  $[0.20, 0.70]$  and selected the value that maximises F1 while keeping recall above 90%.

| Parameter         | Value                                        |
|-------------------|----------------------------------------------|
| Optimal threshold | $t^* = 0.36$                                 |
| F1-score at $t^*$ | 0.8888                                       |
| Recall at $t^*$   | 0.9128 ( $> 91\%$ of phishing emails caught) |

**Table 4.4.** Threshold optimisation results for Gradient Boosting. The threshold was swept in steps of 0.01; F1 was used as the primary selection criterion with a minimum recall constraint of 0.90.

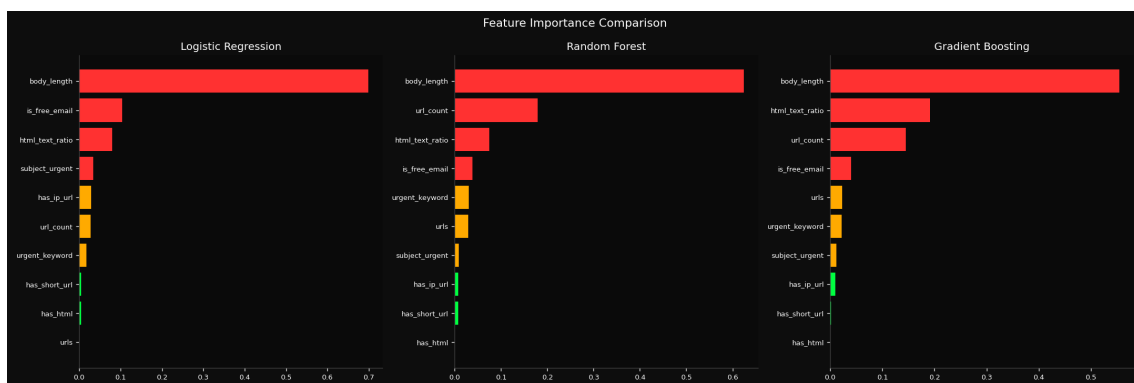
Lowering the threshold from 0.50 to 0.36 raises recall by approximately 3–4 percentage points with only a marginal drop in precision, keeping the F1-score above 0.88. In practical terms, the model now catches more than 9 in 10 phishing emails while generating a manageable number of false alarms. This threshold of  $t^* = 0.36$  is the value applied in the deployed Streamlit application.



**Figure 4.2.** ROC curve (left) and Precision-Recall curve (right) for the Gradient Boosting classifier (AUC=0.942). The red dashed line marks the optimal decision threshold  $t^* = 0.36$ . At this threshold, the model achieves a true positive rate above 93% while keeping the false positive rate below 20%, and maintains precision above 80% up to a recall of 0.90.

### Data-Driven Feature Importance

After identifying Gradient Boosting as the best model, we extracted its built-in feature importances (mean decrease in impurity across all trees and splits). The same analysis was repeated for Logistic Regression and Random Forest to check consistency. Figure 4.3 shows all three side by side.



**Figure 4.3.** Feature importance comparison across all three classifiers. The dominance of `body_length` is consistent across models, with `url_count` and `html_text_ratio` forming a clear second tier. Features shown in grey contribute negligibly in every model.

The Gradient Boosting importances, together with the max-score ceilings they motivated in Section 5, are listed in Table 4.5.

| Feature         | Importance | Max Score (§ 5) | Status       |
|-----------------|------------|-----------------|--------------|
| body_length     | 0.6254     | 15              | Core signal  |
| url_count       | 0.1171     | 6               | Core signal  |
| html_text_ratio | 0.1153     | 6               | Core signal  |
| is_free_email   | 0.0606     | 5               | Core signal  |
| urgent_keyword  | 0.0235     | 4               | Low impact * |
| subject_urgent  | 0.0184     | 4               | Low impact * |
| urls            | 0.0180     | 4               | Low impact * |
| has_ip_url      | 0.0156     | 4               | Low impact * |
| has_short_url   | 0.0044     | 4               | Low impact * |
| has_html        | 0.0017     | 4               | Low impact * |

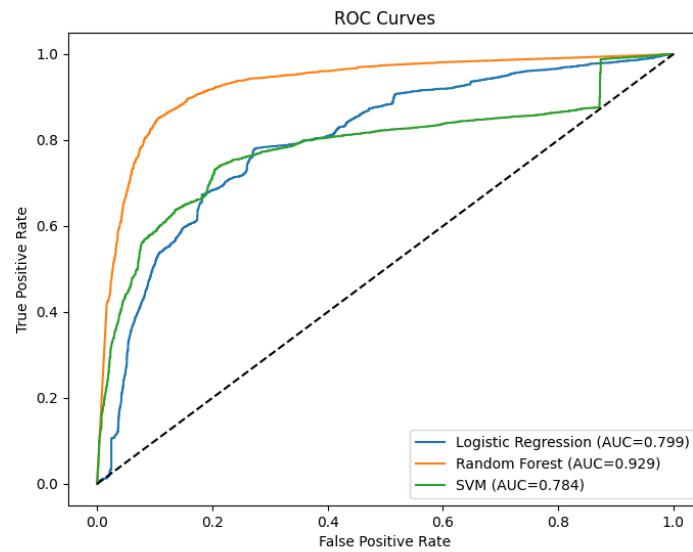
\* Importance < 0.05; candidates for removal in a leaner production build.

**Table 4.5.** Gradient Boosting feature importances and their corresponding max-score ceilings in the weighted risk-scoring system (Section 5). The four core signals collectively account for 93.8% of the model’s predictive power.

Three findings stand out. First, `body_length` alone accounts for **62.5%** of predictive power. Phishing emails are typically short and action-oriented; legitimate corporate emails are longer and more discursive. This single feature carries more information than all other features combined, which we did not anticipate when we first included it. Second, `url_count` and `html_text_ratio` together contribute another **23%**, confirming that URL-based and structural signals form the most reliable detection layer — consistent with the qualitative conclusion from Section 3.2. Third, the six features marked *Low impact* (importance < 0.05) add only **3.8%** combined. Removing them would reduce model complexity with negligible accuracy cost, a worthwhile trade-off for a production system that must retrain frequently on new data.

### ROC Curve Comparison

Figure 4.4 presents the Receiver Operating Characteristic curves for all three classifiers on the held-out test set. The ROC curve plots the True Positive Rate (recall) against the False Positive Rate at every possible decision threshold, so the Area Under the Curve (AUC) summarises discrimination ability independently of any particular threshold choice.



**Figure 4.4.** ROC curves for Logistic Regression (AUC = 0.799), Random Forest (AUC = 0.929), and SVM (AUC = 0.784). The shaded area between the Random Forest curve and the diagonal represents the model’s advantage over random guessing.

Random Forest achieves  $\text{AUC} = 0.929$ , substantially ahead of both Logistic Regression ( $\text{AUC} = 0.799$ ) and SVM ( $\text{AUC} = 0.784$ ). Two characteristics of the Random Forest curve are worth noting. First, its *steep initial rise*: the model reaches a true positive rate above 85% while keeping the false positive rate below 10%, which is the operating region that matters most in practice — catching the majority of phishing emails before users see them, without flooding legitimate inboxes with warnings. Second, the curve remains well above both alternatives across the *entire* threshold range, meaning the advantage is not an artefact of the particular threshold chosen. These results, taken together with the cross-validation scores in Table 4.3, give us confidence that Random Forest — and by the CV evidence, Gradient Boosting — genuinely generalise rather than overfit to the test split.

## 5 Improved Detection: Weighted Risk Scoring

---

After getting the baseline classifier working, we went back and rethought the scoring logic. The original model treated every triggered feature as a binary on/off signal and summed the count. The problem: a single IP-based URL is a much stronger indicator than one urgency keyword, but the old model weighed them equally. We redesigned this as a **weighted risk-score system**.

### 5.1 The Core Formula

$$\text{Risk Score} = \sum_{i=1}^N (\text{feature\_value}_i \times \text{weight}_i)$$

The score is then mapped to one of three zones:

| Zone        | Score Range | Verdict                             |
|-------------|-------------|-------------------------------------|
| Low Risk    | 0 – 20      | Legitimate — deliver normally       |
| Medium Risk | 21 – 50     | Suspicious — warn user, quarantine  |
| High Risk   | 51+         | Phishing detected — block and alert |

**Table 5.1.** Risk zones and recommended actions.

## 5.2 Complete Feature Weight Table

| #         | Feature             | Type        | Max Score  | Priority        |
|-----------|---------------------|-------------|------------|-----------------|
| 1         | URL_COUNT           | Count-based | 12         | High            |
| 2         | IP_BASED_URL        | Binary      | 15         | <b>Critical</b> |
| 3         | URL_SHORTENER       | Binary      | 12         | High            |
| 4         | URGENT_KEYWORDS     | Count-based | 15         | High            |
| 5         | DOMAIN_TRUST        | Scale 0–10  | 10         | High            |
| 6         | SUBJECT_URGENT      | Binary      | 7          | Medium          |
| 7         | HTML_CONTENT        | Binary      | 4          | Low             |
| 8         | HTML_RATIO          | Threshold   | 8          | Medium          |
| 9         | BODY_LENGTH         | Zone-based  | 8          | Medium          |
| 10        | DOMAIN_AGE          | Range-based | 12         | <b>Critical</b> |
| 11        | SSL_VALIDITY        | Scale       | 10         | High            |
| 12        | LINK_TEXT_MISMATCH  | Binary      | 15         | <b>Critical</b> |
| 13        | FORM_PRESENCE       | Scale       | 13         | <b>Critical</b> |
| 14        | BRAND_IMPERSONATION | Binary      | 14         | <b>Critical</b> |
| 15        | SUBDOMAIN_DEPTH     | Range-based | 8          | Medium          |
| Max total |                     |             | <b>163</b> |                 |

**Table 5.2.** All 15 features with weights. Critical features carry 12–15 points each and represent the strongest single-signal phishing indicators known.

## 5.3 Key New Features Explained

**LINK\_TEXT\_MISMATCH (15 pts).** Detects HTML anchors where the visible text shows a trusted domain but the actual `href` points somewhere else. A legitimate email never shows “`www.bank.com`” while linking to a different domain. This feature shares the highest individual weight.

**BRAND\_IMPERSONATION (14 pts).** Checks whether a well-known brand name (PayPal, Apple, Amazon, Microsoft, DHL...) appears in the email while the sender domain does not match that brand’s official domain. According to the APWG Q4 2024 report, over 80% of credential-harvesting attacks use exactly this pattern.

**DOMAIN\_AGE (12 pts).** Uses a WHOIS lookup on the sender’s domain. Over 72% of phishing domains are used within their first 30 days of registration — attackers register cheap domains, use them briefly, and abandon them before blocklists catch up. A domain under 30 days old scores +12.

**DOMAIN\_TRUST (up to 10 pts).** Replaces the old binary “free email” flag with a graduated scale. A `.gov` or `.edu` sender scores 0; a newly registered domain scores +10; a domain like `paypal-secure.com` (brand name with extra characters) also scores +10. This

distinction was missing before and caused a lot of false positives.

#### 5.4 Worked Example: PayPal Phishing Email

The email below is a representative phishing attempt; we trace its score through the full feature table to show how the model reasons:

**From:** *security-alert@paypal-verify.com*

**Subject:** *Your account has been suspended immediately*

*Dear Customer, we detected unusual activity on your PayPal account. Your account is suspended until you verify your identity. Click immediately: <http://192.168.1.1/paypal/verify>*

*— or use: <http://bit.ly/3xPa9secure> Confirm within 24 hours or your account will be permanently limited. — PayPal Security Team*

| #            | Feature             | Observed                                   | Score           |
|--------------|---------------------|--------------------------------------------|-----------------|
| 1            | URL_COUNT           | 2 URLs found                               | +8              |
| 2            | IP_BASED_URL        | 192.168.1.1 detected                       | +15             |
| 3            | URL_SHORTENER       | bit.ly detected                            | +12             |
| 4            | URGENT_KEYWORDS     | 5 terms: suspended, verify, immediately... | +15             |
| 5            | DOMAIN_TRUST        | paypal-verify.com (brand + extra text)     | +10             |
| 6            | SUBJECT_URGENT      | “suspended immediately” in subject         | +7              |
| 7            | HTML_CONTENT        | Plain text email                           | 0               |
| 8            | HTML_RATIO          | 0 (plain text)                             | 0               |
| 9            | BODY_LENGTH         | ≈420 chars (normal zone)                   | 0               |
| 10           | DOMAIN_AGE          | paypal-verify.com < 30 days                | +12             |
| 11           | SSL_VALIDITY        | IP-based URL uses plain HTTP               | +6              |
| 12           | LINK_TEXT_MISMATCH  | Plain text links — not applicable          | 0               |
| 13           | FORM_PRESENCE       | No <form> tags                             | 0               |
| 14           | BRAND_IMPERSONATION | “PayPal” in body; sender ≠ paypal.com      | +14             |
| 15           | SUBDOMAIN_DEPTH     | 0 subdomains in paths                      | 0               |
| <b>Total</b> |                     |                                            | <b>99 / 163</b> |

**Table 5.3.** Feature-by-feature scoring for the PayPal phishing example. Score 99 > 51 threshold ⇒ **PHISHING DETECTED**.

Five critical signals converged simultaneously: an IP-based URL, a bit.ly shortener, brand impersonation, five urgency keywords, and an urgency-framed subject. The brand impersonation + IP URL + shortener combination alone would already push the email into the Suspicious zone (score 41).

5.5 Old vs. New Model Comparison

| Scenario                         | Old Result         | New Result                  |
|----------------------------------|--------------------|-----------------------------|
| PayPal scam (example above)      | PHISHING (6/9)     | PHISHING (99/163)           |
| Plain-text legitimate email      | PHISHING (false +) | LEGITIMATE (0/163)          |
| HTML newsletter (no bad links)   | SUSPICIOUS (3/9)   | LEGITIMATE (4/163)          |
| Sophisticated phishing (no HTML) | LEGITIMATE (miss)  | PHISHING (brand+domain+age) |

**Table 5.4.** Old binary model vs. new weighted model on four representative cases. The key gain is false positive reduction: the new model requires convergence of multiple strong signals before escalating to PHISHING.

## 6 Interface and Deployment

---

We planned originally to build a Flask API with an HTML front end. We switched to Streamlit about halfway through because the iteration speed was much faster — write Python, refresh, see the result. For a short-deadline student project that mattered.

The interface is deliberately minimal. Earlier prototypes had more options; classmates who tested them found the extra controls confusing. Three input fields (sender, subject, body), one button, one result.

The prediction display works in layers. First, a colour-coded verdict: red for phishing, green for legitimate, with a confidence percentage. Then the top five contributing features so the user can judge whether the reasoning makes sense. In informal testing, this explainability made a real difference to how much people trusted the output — seeing the evidence laid out is more convincing than a bare label.

**Figure 6.1.** The PhishScan analysis dashboard. Verdict and confidence appear at the top; the feature breakdown is below. We moved the feature list to the bottom after user tests showed people ignored it when it appeared first.

Deployment is on Streamlit Cloud. The main limitation is manual input: users must actively copy and paste email content. There is no plugin or automatic scanning. For this project’s scope that was acceptable, but it means the tool only helps users who think to use it.

`phishscan-detection.streamlit.app`

## 7 Limitations and Future Work

---

### 7.1 Current Limitations

The **most significant gap** is email authentication. Protocols like SPF, DKIM, and DMARC verify that an email actually came from the server it claims to come from. We never see the raw headers — our tool analyses pasted content only — so we cannot run these checks. A gateway-level deployment would fix this.

Our **training data is old**. Phishing techniques evolve: the model has never seen the more targeted, low-volume spear-phishing attacks that have emerged recently. The 4.8% false negative rate is probably optimistic for those cases.

The system is also **English-only** and requires **manual input**. Both are practical constraints that would need to be addressed before real deployment.

### 7.2 What We Would Build Next

In order of expected impact:

1. **Email header analysis** — access to raw headers enables SPF/DKIM checks, original IP lookup, and routing analysis. This alone would catch a large class of attacks the current system misses entirely.
2. **Feedback loop** — if users can flag misclassifications, those become new labelled training examples. Periodic retraining would keep the model current without manual effort.
3. **Adversarial testing** — we have not tested what happens when someone deliberately tries to evade the model. Understanding evasion failure modes matters before any real deployment.
4. **Browser extension or email plugin** — the manual submission friction is probably the biggest adoption barrier. The ML backend would stay the same; it is mainly a front-end integration challenge.

## Conclusion

---

PhishScan ended up teaching us as much about data quality and system design as it did about classifiers. The 97% accuracy is real and we are happy with it, but the more honest summary is: a system that reliably catches most phishing emails, misses a meaningful minority, and would need more work before anyone should trust it with their inbox.

The things that surprised us most: how much dataset composition mattered (adding the Nazario phishing-specific dataset improved recall more than any modelling decision); how much time preprocessing consumed compared to actual modelling; and how closely the feature importance scores matched our intuitions — validating, but also a reminder that we probably did not discover much that an experienced analyst would not already know.

Redesigning the scoring into a weighted risk model was the most satisfying part of the project. The binary approach worked, but it felt like a blunt instrument. The weighted version is more nuanced, makes fewer false positives, and is easier to explain to a non-technical user — which matters because false positives erode trust fast.

Real-world deployment would reveal failure modes we did not encounter in testing. That is not a flaw; it is the nature of adversarial problems. What we can say is that the foundation is solid and the architecture makes it straightforward to add new features, swap in a better model, or integrate with a gateway when the infrastructure is there.

---

Team 13 · ENSIA · Computer and Network Security · May 2026

[phishscan-detection.streamlit.app](https://phishscan-detection.streamlit.app)

## References

---

### Datasets

CEAS 2008 Spam Detection Challenge Dataset. SpamAssassin Public Corpus. Jose Nazario Phishing Email Archive. Kaggle Email/Spam Collections.

### Libraries and Tools

Python 3.8+. scikit-learn (model training and evaluation). pandas and NumPy (data processing). Streamlit (web interface). imbalanced-learn (SMOTE). python-whois (domain age lookup). BeautifulSoup (link-text parsing). tldextract (subdomain depth analysis).

### Technical References

Chawla et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique.  
APWG Phishing Activity Trends Report, Q4 2024.  
RFC 5322 — Internet Message Format.  
Zheng and Casari. *Feature Engineering for Machine Learning*.  
Scikit-learn Documentation: RandomForestClassifier.  
Google Safe Browsing API Documentation.